Reading Round-Up Dashboard

Handover Documentation

Table of Contents

1. Executive Summary 3

2. Mental Model of the System 3

3. Technical Design 4

4. Article Pipeline Database 14

5. Local Setup 15

6. Usage guide 16

7. Google Cloud and Service account monitoring 16

8. Maintenance and Operations 19

9. Troubleshooting Guide 20

10. Known Limitations 20

11. Next Steps 21

12. Future Projects 23

1. Executive Summary

Reading Roundup dashboard helps the team collect articles from configured publication lists, summarize them, save structured results, generate PDF artifacts, validate source quality, and run trend analysis.

The most important architectural shift from the earlier TaterTot version is the backend boundary. In App-revamp (now merged to master), the frontend is responsible for user interaction, while the backend owns authentication, GitHub workflow dispatches, Google credentials, run state, source validation, artifact download, and trend execution. This reduces the risk of exposing privileged tokens in browser-delivered code and gives the backend a single place to enforce auth, cooldowns, status tracking, and logging.

The current system is not just a scraper plus UI. It is closer to a full operational pipeline: source configuration, source validation, collection, AI summarization, storage, reporting, dashboard display, and trend analysis are all connected.

2. Mental Model of the System

Think of the system as a controlled pipeline wrapped in a dashboard. The dashboard is the control surface. The backend is the gatekeeper and orchestrator. GitHub Actions runs long jobs. Google Sheets stores operational data. Render hosts the backend service that the dashboard talks to.

In a normal collect/summarize run, the user signs in, selects a source list, optionally provides keyword overrides, and starts the pipeline. The frontend sends the request to the FastAPI backend. The backend validates the session, checks that another run is not already active, enforces cooldown rules, and dispatches the `Collect & Summarize Articles` GitHub Actions workflow. The workflow installs backend dependencies, writes Google credentials from repository secrets, runs `backend/pipeline_runner.py`, uploads the generated PDF as a GitHub artifact, and cleans up credentials.

The pipeline itself initializes Google Sheets storage, loads the selected source list, collects article candidates from RSS feeds and sitemaps, extracts article content, summarizes articles with the configured Hugging Face model, writes rows to Google Sheets, generates a PDF, and saves run metadata.

For trend analysis, the dashboard calls the backend trend trigger endpoint. The backend dispatches `trend-analysis.yml` with inputs such as list name, window mode, date range, baseline weeks, topic, and trend run id. The workflow runs `backend/run_trend_analysis.py`, which filters articles by source-list host, computes trend rows, writes them to the `Trend Signals`sheet, and updates trend metadata.

3. Technical Design

3.1 High-Level Architecture

The frontend lives in frontend/and is built with React 19, Vite 7, Tailwind CSS 4, and lucide-react icons. Its job is to handle login, source list selection, pipeline controls, progress display, article results, starred summaries, source-list management, and trend analysis views. It should not own privileged integrations.

The backend lives in backend/and is built with FastAPI. Its main entry point is backend/trigger_service.py. This file handles auth, CORS, security headers, GitHub workflow dispatch, run state, websocket status updates, artifact download, source-list management, source metadata validation jobs, starred summaries, and trend endpoints.

The pipeline is mostly coordinated by backend/pipeline_runner.py. It instantiates CustomArticleCollector from backend/AgentCollector.py, ArticleSummarizer from backend/AgentSumm.py, and GoogleSheetsDBfrom backend/google_storage.py. Collection, summarization, storage, PDF generation, and metadata updates happen in that order.

The system also has three GitHub Actions workflows under .github/workflows/:

collect-articles.ymlis manually triggered and runs the main collection/summarization pipeline. It accepts test_mode, keywords, topic, and list_name. It has global concurrency protection with cancel-in-progress: false, uses Python 3.10, retries dependency installation, runs the pipeline with a 110 minute command timeout inside a 120 minute job timeout, sets SUMMARIZER_MODEL=google/flan-t5-base, uploads the PDF as a Reading-Roundup-YYYY-MM-DD.pdfartifact, and deletes credentials.jsonafterward.

trend-analysis.ymlis manually triggered and runs trend analysis. It accepts target_week_key, topic, window_start_date, window_end_date, baseline_weeks, trend_run_id, window_mode, and list_name. It uses Python 3.11, logs run context, writes Google credentials, runs run_trend_analysis.py, and cleans credentials afterward. It no longer auto-runs after the collect workflow.

deploy-frontend.ymldeploys the frontend to GitHub Pages. It currently watches pushes to the master branch. If the repo has now been merged into masterand future production changes happen on master, this workflow should be updated so frontend deployments trigger from the correct branch.

3.2 Important Files and Responsibilities

backend/trigger_service.pyis the backend API service. It defines login, refresh, logout, pipeline trigger/status, websocket status, latest artifact, direct artifact ZIP download, current/latest trend result retrieval, trend trigger, trend run status, source list creation/listing, source metadata progress, and source

metadata report endpoints. It also adds security headers and a CORS allowlist.

backend/pipeline_runner.pyis the main execution orchestrator for the article pipeline. It reads topic, source list, and keyword override environment variables, initializes the collector and summarizer, saves pipeline progress to Google Sheets metadata, runs collection, runs summarization, saves articles, generates the PDF, skips Drive upload in the current active path, saves GitHub run metadata, and records stats.

backend/AgentCollector.pycontains the scraping and article candidate logic. It supports finance and luxury source sets, can load active sources from the Source ListsGoogle Sheet, prefers sitemap discovery before RSS fallback, rotates user agents, rate-limits requests, uses stronger request clients when available, filters invalid URLs, scores candidates by weighted keywords, extracts full content, and skips sources that repeatedly return 401, 403, placeholders, or malformed URLs.

backend/AgentSumm.pycontains the Hugging Face summarizer and author extraction logic. It resolves the model from a constructor argument, then SUMMARIZER_MODEL, then facebook/bart-large-cnn. In the current collect workflow, SUMMARIZER_MODELis explicitly set to google/flan-t5-base, so the production GitHub Actions path uses FLAN-T5 Base. Because the model name contains flan, the code uses the promptable path and wraps article text with a finance/media-monitoring prompt before summarization.

backend/google_storage.pymanages Google Sheets and Drive-related helper operations. It writes article rows, retrieves recent articles, manages metadata, saves pipeline progress and stats, stores GitHub artifact info, manages starred summaries, and contains older Drive upload helper code. In the current pipeline path, Drive upload is disabled and GitHub artifact ZIP download is the active PDF access path. The Drive helper still contains public sharing behavior if re-enabled, so it should be reviewed before any future Drive-based report distribution is turned back on.

backend/publication_metadata_pipeline.pyvalidates source configuration. It reads active rows from the Source Listssheet, discovers sitemap candidates from robots.txtand common sitemap paths, validates sitemap XML, validates RSS feeds with feedparser, writes detailed rows to Source Validation Details, and updates active status based on whether the source has usable metadata.

backend/run_trend_analysis.pyruns trend analysis in GitHub Actions. It reads rows from the article sheet, filters them to hosts in the selected source list, computes trends using trend_analyzer.py, writes rows to Trend Signals, inserts a _NO_TRENDS_sentinel row when no trends pass thresholds, and updates trend metadata such as latest run id, status, rows written, topic, window mode, and date window.

frontend/src/services/githubAPI.jsis now a backend API client, despite the legacy filename. It does not directly call GitHub Actions from the browser. It handles login, token refresh, pipeline trigger/status, artifact download, stars, source lists, source metadata jobs, trend triggers, and trend result polling by calling the FastAPI backend.

frontend/src/services/googleSheetsAPI.jsstill reads Google Sheets from the browser for dashboard article/result display. It uses VITE_GOOGLE_SHEET_IDand VITE_GOOGLE_API_KEY. This is read-only display access, not privileged workflow dispatch. However, the frontend .env.examplestill contains older VITE_GITHUB_*variables. Those should be treated as legacy documentation and removed or replaced so future interns do not put GitHub tokens into frontend env vars.

frontend/src/components/ManageTopicsPage.jsxis the source-list management UI. It lets users enter a list name and one source per line in base_url, rss_urlformat, save those rows through the backend, run metadata validation, poll metadata progress, and view a validation report.

frontend/src/components/TrendAnalysisView.jsxis the trend analysis UI. It loads source lists, lets users select a list and window mode, triggers trend analysis through the backend, polls run status, and displays latest/by-run results through TrendResultsList.

3.3 Suggested Reading Order

Start with backend/trigger_service.pybecause it explains how the dashboard reaches the backend and how protected operations are exposed. Then read backend/pipeline_runner.pyto understand the collect/summarize/store/report sequence. After that, read backend/AgentCollector.py, backend/AgentSumm.py, and backend/google_storage.py.

Once the main pipeline is clear, read backend/publication_metadata_pipeline.py, backend/run_trend_analysis.py, and backend/trend_analyzer.py. On the frontend side, start with frontend/src/services/githubAPI.js, frontend/src/components/SummariesView.jsx, frontend/src/components/ManageTopicsPage.jsx, frontend/src/components/TrendAnalysisView.jsx, and frontend/src/components/LoadingScreen.jsx.

For deployment behavior, read all workflows in .github/workflows/, especially collect-articles.yml, trend-analysis.yml, and deploy-frontend.yml.

3.4 Security and Data Boundaries

The browser must be treated as untrusted. GitHub tokens, Google service account credentials, JWT secrets, and app passwords should never be placed in frontend code or frontend environment variables. The frontend should call authenticated backend endpoints, and the backend should perform privileged actions.

The frontend triggers /pipeline/triggeror /trends/trigger, and the backend uses GITHUB_TOKENserver-side. The same pattern applies to source metadata jobs and artifact download. The browser should never need a GitHub Personal Access Token.

The backend uses JWT access tokens and refresh cookies. Relevant settings include APP_LOGIN_PASSWORD, JWT_SECRET, JWT_EXPIRES_SECONDS, REFRESH_EXPIRES_SECONDS, REFRESH_COOKIE_SECURE, and REFRESH_COOKIE_SAMESITE. CORS is explicitly allowlisted for local development, Render, GitHub Pages, and rrd.claireadler.com. Do not broaden CORS casually during debugging because doing so weakens the deployment boundary.

The current frontend .env.example still shows legacy VITE_GITHUB_TOKEN, VITE_GITHUB_OWNER, and VITE_GITHUB_REPO variables. That example should be cleaned up. For the current frontend, the key production value is VITE_PIPELINE_API_BASE; Google Sheet ID/API key may still be used for read-only article display, but GitHub tokens should stay backend-only.

3.5 Deployment and Operations on Render

Render hosts the FastAPI backend. When dashboard actions fail, Render logs are usually the first place to look because they show backend request handling, auth failures, GitHub dispatch failures, Google access errors, trend trigger errors, and status polling issues.

In Render, open the backend service. The Dashboard page shows service health and the current deploy. The Logs page shows live and historical backend logs. The Environment page stores production secrets and runtime settings. The Deploys page shows deploy history and lets you manually redeploy after environment changes. The Settings page contains service type, build command, start command, region, and instance configuration.

The most important Render env vars are GITHUB_OWNER, GITHUB_REPO, GITHUB_WORKFLOW, GITHUB_REF, GITHUB_TOKEN, GOOGLE_SHEET_ID, GOOGLE_CREDENTIALS, APP_LOGIN_PASSWORD, and JWT_SECRET. Optional but useful env vars include PIPELINE_COOLDOWN_SECONDS, GITHUB_TERMINAL_POLL_SECONDS, SOURCE_CONFIG_SHEET, SOURCE_REPORT_DETAIL_SHEET, STARRED_SHEET_NAME, and ALLOW_AUTO_TREND_TRIGGER.

GITHUB_REF is a common source of production bugs. If Render points to an old branch, the backend will dispatch workflows from that old branch even if the latest workflow file on GitHub looks correct. This caused the kind of 422 Unexpected inputs provided error seen when the backend sent trend inputs that were not present in the workflow definition on the dispatched ref. If workflow inputs look correct but dispatch still fails, check GITHUB_REF in Render, update it to the active branch, and redeploy the service.

3.6 Web Scraping Architecture

Purpose

The scraping layer lives mainly in `backend/AgentCollector.py`. Its job is to discover article URLs from each configured publication, rank them by relevance, fetch the full article content, and return clean article objects to the summarization pipeline.

The current collector is primarily a sitemap/RSS-driven scraper with anti-blocking request handling, candidate relevance scoring, and Newspaper3k/BeautifulSoup-based full-content extraction.

Main Challenge

Modern publications are difficult to scrape because they often use Cloudflare, rate limiting, bot detection, paywalls, changing HTML layouts, and inconsistent feed support. Because of this, App-revamp does not depend on one scraping technique. It uses layered request handling and fallback parsing so one failing publication does not stop the whole run.

Request Client Priority

Before fetching pages, the collector chooses the strongest available HTTP client in this order:

1. curl_cffi
2. cloudscraper
3. requests

curl_cffi is preferred because it better mimics browser-like TLS/HTTP behavior. cloudscraper is used as the next fallback for Cloudflare-protected pages. Plain requests is used only if the stronger clients are unavailable.

This keeps the pipeline runnable in different environments while using stronger anti-blocking behavior when possible.

User-Agent Rotation and Rate Limiting

Each request gets a realistic browser user-agent selected from a rotating list. The collector also applies delays between requests and adds random timing variation.

This reduces repetitive request patterns that can trigger anti-bot systems. The goal is stable collection, not aggressive scraping.

Article Discovery

The current discovery flow is sitemap-first, then RSS fallback.

For each publication, the collector checks the configured sitemap_url first. If sitemap discovery produces

no candidates or too few useful candidates, it falls back to configured RSS feeds. Later, if fewer than three usable full articles were extracted, it may retry RSS candidates again to fill the gap.

This means source quality depends heavily on good sitemap_url and rss_url values in the Source Lists sheet.

Keyword based Candidate Filtering and Scoring

Discovered URLs are sanitized, deduplicated, and filtered before full extraction. The collector removes invalid URLs, irrelevant URLs, and source-specific unwanted paths where possible.

Each candidate receives a relevance score based on topic-specific keyword weights. Finance and luxury have different keyword profiles, and dashboard keyword overrides can customize a run. Higher-scoring candidates are attempted first, so the pipeline spends time on the articles most likely to be useful.

Full Article Extraction

After candidate ranking, the collector fetches each article URL and attempts to extract the full article body. Newspaper3k is the main article parsing tool, with BeautifulSoup used in supporting/fallback contexts.

The extracted article object includes title, URL, publication, full content, candidate source, relevance score, and author information when available. Final author/journalist handling is split between the collector and AgentSumm.py; the summarization step can update the author field before saving rows to Google Sheets.

Failure Handling

The collector tracks repeated source-level failures such as:

- HTTP 401 responses
- HTTP 403 responses
- HTTP 429/rate-limit responses
- malformed URL or curl errors
- placeholder pages
- short extracted text
- low relevance score

If a source keeps failing, the collector stops wasting time on that source and moves to the next publication. This is important because one broken publication should not block the full pipeline.

Important Limitation

A source can pass RSS/sitemap validation and still fail during article extraction. For example, a sitemap may be valid but point to paywalled pages, irrelevant URLs, placeholders, or pages with unusable article text.

The source metadata pipeline confirms whether discovery endpoints are available. The scraper still has to validate whether the discovered article pages are usable. That is why metadata validation is necessary but not sufficient.

3.7 Summarisation Pipeline

Purpose

The summarization pipeline is implemented in backend/AgentSumm.py. and orchestrated from backend/pipeline_runner.py.. Its job is to take the full article text extracted by the collector, generate a concise summary, finalize the author/journalist field, and return a clean article object that can be saved to the Articles tab in the Article Pipeline Database.

The summarizer runs after article collection and before database saving. Raw full_content is kept only long enough to generate the summary. After summarization succeeds, pipeline_runner.py removes full_content before writing the article row to Google Sheets.

Current Model Configuration

The current production GitHub Actions workflow configures the summarizer with:

```
SUMMARIZER_MODEL=google/flan-t5-base
```

This means production collect/summarize runs use FLAN-T5 Base, not the old default BART model.

In AgentSumm.py, the model is loaded through Hugging Face:

```
self.summarizer = pipeline("summarization", model=resolved_model)
```

The model is resolved in this order:

- constructor argument passed into ArticleSummarizer
- SUMMARIZER_MODEL environment variable
- fallback default: facebook/bart-large-cnn

So if SUMMARIZER_MODEL is missing, the code still falls back to BART. But in the current collect workflow, SUMMARIZER_MODEL is explicitly set to google/flan-t5-base.

Why FLAN-T5 Base Is Used

FLAN-T5 Base is instruction-tuned, which makes it better suited for prompt-style summarization than

plain BART. Since the pipeline needs summaries tailored to PR/media monitoring, the ability to prepend an instruction is useful.

The current summarizer prompt asks the model to summarize finance articles for PR/media monitoring and focus on:

- market impact
- institutions and companies
- regulation and policy
- key numbers
- one concise paragraph

This is more controllable than generic summarization, especially when the same pipeline may be used for different editorial or business topics.

Promptable Model Path

AgentSumm.py checks whether the selected model name contains t5 or flan:

```
self.is_promptable = "t5" in resolved_model.lower() or "flan" in resolved_model.lower()
```

Because google/flan-t5-base contains both flan and t5, the code uses the promptable path.

In that path, the summarizer builds an instruction prompt before the article text:

```
Summarize this finance article for PR/media monitoring.
Focus on market impact, institutions/companies, regulation/policy, and key numbers.
Write one concise paragraph.

Article:
{input_text}
```

This prompt is then sent to the Hugging Face summarization pipeline.

FLAN-T5 Base Runtime Settings

For FLAN/T5-style models, the current code uses:

```
max_length = 260
min_length = 100
do_sample = False
truncation = True
num_beams = 4
length_penalty = 1.0
early_stopping = True
```

These settings make output deterministic and more consistent across runs. Beam search explores multiple candidate summaries, while do_sample=False avoids random generation. The longer max/min length compared with the old BART settings is intended to produce more substantive paragraph summaries.

Input Truncation

Before summarization, article content is truncated to:

```
input_text = article_content[:4000]
```

This keeps inference manageable in GitHub Actions and avoids exceeding model limits. The tradeoff is that very long articles may lose later context. In most news articles, the opening sections usually contain the highest-value facts, so this is acceptable for the current pipeline.

Author Handling

The summarization step also finalizes author/journalist data. The collector may already pass an author value, but AgentSumm.py includes additional author extraction helpers using:

- JSON-LD structured data
- Newspaper3k author fields
- regex patterns such as By Name
- fallback to Unknown

After summarization, pipeline_runner.py updates both author and journalist fields from the summary object before saving to Google Sheets.

Alternatives Considered

BART (facebook/bart-large-cnn)-

BART was the original/default model and is still the fallback in code. It produces strong abstractive summaries and is pre-trained on CNN/DailyMail-style news data. However, it is less naturally instruction-following than FLAN-T5, so domain-specific prompt control is weaker.

T5-small-

T5-small was considered because it is lightweight and faster, but summaries tend to be too generic and lower quality for PR/media monitoring. It is easier to run, but not strong enough for the level of summary quality expected by users.

Pegasus-XSum-

Pegasus-XSum is strong for abstractive summarization, but it can be slow and less practical in the GitHub Actions environment. Runtime matters because the pipeline processes many articles per run.

GPT-3.5/API-based summarization-

An API-based LLM could likely produce higher-quality, more flexible summaries. However, it introduces cost, privacy, and client-data concerns. The current local Hugging Face approach keeps summarization inside the controlled pipeline environment.

Current Limitation

The summarization prompt is currently fixed in code. Users cannot yet provide custom summarization instructions from the dashboard. This is a high-priority future improvement because different topics may need different summary angles, such as finance impact, luxury brands, jewelry, executives, celebrities, or outreach relevance.

3.8 Adding New Publications and Source Lists

New publications can be added through the source-list workflow whenever possible. The source configuration sheet expects `list_name`, `publication`, `base_url`, `sitemap_url`, `rss_url`, `active`, and `date_added`. The dashboard's “Add New Category/Topic” page lets the user create a list by entering a list name and source rows in `base_url`, `rss_url`format.

The user starts by gathering the publication's base URL and looking for an RSS feed. RSS feeds are preferred because they usually provide cleaner and more stable article discovery than scraping arbitrary index pages. At the moment the user uses https://lighthouseapp.io/ to discover rss feed URLs.

After adding sources, they run the metadata check. The backend starts a metadata job, polls progress, and writes detailed validation rows to `Source Validation Details`. The automation reviews whether each source has a valid sitemap, valid RSS feed, and active status after validation, and generates a report and displays it after the completion of the run.

A valid sitemap does not guarantee a useful source. Some sitemaps contain irrelevant URLs, paywalled placeholders, old content, or pages that fail article extraction. Metadata validation should be followed by a small collection run or careful inspection of pipeline logs to confirm that the source produces real, relevant article candidates.

3.9 Trend Analysis

Trend analysis is now a first-class workflow. It compares a selected current window against a historical baseline and writes ranked trend signals to Trend Signals. The current workflow thresholds are TREND_MIN_MENTIONS=2, TREND_MIN_LIFT=1.2, TREND_MIN_PUBLICATIONS=1, and TREND_TOP_N=25.

The trend script filters articles by source-list host. That means the URLs in the article sheet must match hosts derived from active rows in Source Lists. If a list has no active hosts or if article URLs do not match the configured hosts, trend analysis can load zero articles and produce the _NO_TRENDS_ sentinel row.

BERTopic is included in backend dependencies as a possible future direction for semantic topic modeling, but the current production trend-analysis workflow does not use BERTopic. The active implementation relies on article text features, keyword extraction/scoring, publication coverage, and historical baseline comparison. A future version could use BERTopic to cluster articles into semantic themes before ranking trend signals.

If trend results look empty, check the selected list, whether the list has active rows, whether the article URLs match the list hosts, whether the chosen date window contains article data, and whether thresholds are too strict for the available sample size.

4 Article Pipeline Database

The “Article Pipeline Database” is a Google Sheets workbook used as the operational database for the Reading Roundup pipeline. The backend connects to it through the GoogleSheetsDB class in backend/google_storage.py, using a Google service account. In production, the sheet is selected through the GOOGLE_SHEET_ID environment variable, and credentials are provided through GOOGLE_CREDENTIALS or a temporary credentials.json file created during GitHub Actions. The backend uses Google Sheets as the shared source of truth because it is easy for non-technical users to inspect, manually correct, and export without needing a separate database UI.

The main tab for pipeline output is **Articles**. Each successful collect/summarize run appends new rows to this tab. The pipeline does not overwrite the previous run’s article rows; it appends to preserve historical records. The columns currently written are ID, TITLE, URL, PUBLICATION, JOURNALIST, SUMMARY, COLLECTED DATE, and SCORE. These values are assembled in backend/pipeline_runner.py after article collection and summarization. During collection, each article receives an internal ID, title, URL, publication name, extracted full content, and relevance score. During summarization, the full content is sent to the Hugging Face summarizer, the summary is filled in, the author/journalist field is updated, and full_content is removed before saving so the sheet stores the final readable output rather than the raw scraped body.

Rows are saved to the Articles tab only after summarization completes. In pipeline_runner.py, the sequence is: collect article candidates, summarize articles and extract authors, then call self.db.save_articles(summarized_articles). The actual sheet write happens in google_storage.py through self.articles_sheet.append_rows(...). At that moment, the backend adds the current timestamp as COLLECTED DATE and writes the relevance score into the SCORE column. This means the collected date represents when the pipeline saved the article to the database, not necessarily the original article publication date.

The same workbook also stores operational data in other tabs. **Source Lists** defines which publications belong to each topic/list and whether each source is active. **Source Validation Details** stores the results of RSS and sitemap validation runs. **Metadata** stores run-level state such as latest GitHub run number, latest run URL, article count, pipeline phase, progress message, and trend status. **Trend Signals** stores trend-analysis output by run id and time window. Starred Summaries stores user-starred articles for follow-up. **Outreach Drafts** and **Pitching Menu** exist for related outreach workflows.

In short, the sheet is both the article database and the lightweight operations database. The Articles tab stores the actual summarized article outputs, while Metadata, Source Lists, Source Validation Details, and Trend Signals allow the dashboard and backend to coordinate pipeline state, source health, and trend-analysis results.

5 Local Setup

Use local setup for understanding the app and testing small changes. Do not put production secrets into frontend env files. The frontend should know the backend API base URL, while privileged values belong in backend env.

Backend setup:

```
cd master/backend python
-m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt uvicorn
trigger_service:app --reload
```

On macOS/Linux, use `source .venv/bin/activate` instead of the Windows activation command. The backend will need the required environment variables before protected integrations can work.

Frontend setup:

```
cd master/frontend npm
install
npm run dev
```

After both services start, verify login, /pipeline/status, one source list fetch, and one dashboard action. Use the browser Network tab to confirm the frontend is talking to VITE_PIPELINE_API_BASEand not trying to call GitHub directly.

6 Usage Guide

For normal users, the dashboard is the preferred way to run the system. Open the deployed frontend URL, sign in, select the topic/source list, optionally provide keyword overrides, and click the run button. The backend will reject duplicate active runs and enforce cooldown rules. The collect workflow has a 120 minute job timeout, and the current realistic runtime is closer to one and a half hours than the older 3-7 minute prototype estimate.

For technical users, GitHub Actions is the best debugging entry point. Open the repository, go to the Actionstab, select Collect & Summarize Articles, choose the correct branch, and run the workflow manually. The workflow accepts test_mode, keywords, topic, and list_name. Logs will show dependency installation, Google credential setup, pipeline attempts, PDF discovery, artifact upload, and cleanup.

Trend analysis is run from the Trend Analysis dashboard view or directly through the Trend Analysis (Phase 4A) GitHub Actions workflow. The dashboard path calls /trends/triggerwith the explicit trend-run header, and the backend dispatches trend-analysis.yml. The workflow is manual-only and accepts the current window/list inputs. The UI polls /trends/run-statusand then fetches results by run id or latest run.

PDF/report access currently uses GitHub Actions artifacts. The collect workflow uploads the generated PDF as an artifact whose name starts with Reading-Roundup-. The backend has /pipeline/download-latest-artifact, which streams the latest matching artifact ZIP to the browser. Users may still receive a ZIP file rather than a direct standalone PDF, because GitHub Actions artifacts are ZIP archives.

7 Google Cloud and Service Account Monitoring

Purpose

The Google service account is already created and is used by the backend and GitHub Actions to access the Article Pipeline Database in Google Sheets. The service account is what allows the pipeline to write article rows, metadata, source validation results, trend signals, and other operational data without using a personal Google account.

The main things a future intern needs to know are how to verify the service account is still working, where its credentials are used, and what to check when Google Sheets or Drive access fails.

Where the Service Account Is Used

The backend connects to Google Sheets through backend/google_storage.py. The GoogleSheetsDB class authenticates using either:

- a local credentials.json file, or
- the GOOGLE_CREDENTIALS environment variable in GitHub Actions / Render.

In GitHub Actions, the workflow writes the repository secret into a temporary file:
echo '${{ secrets.GOOGLE_CREDENTIALS }}' > credentials.json

Then the pipeline runs with:
GOOGLE_SHEET_ID=${{ secrets.GOOGLE_SHEET_ID }}

After the workflow finishes, credentials.json is deleted during the cleanup step.

How to Monitor the Service Account

Go to Google Cloud Console and open the project that owns the service account. Then go to:
IAM & Admin -> Service Accounts

Find the service account used by the Reading Roundup pipeline. It will usually have a name similar to reading-roundup-bot.

Check the following:

1. The service account still exists.
2. It has not been disabled.
3. It still has a valid JSON key if the system is using key-based auth.
4. The key has not been deleted or rotated without updating GitHub/Render secrets.
5. The service account email still has Editor access to the Google Sheet.

How to Verify Sheet Access

Open the Article Pipeline Database Google Sheet and click Share.
Confirm the service account email is listed with Editor access. The email usually looks like:
reading-roundup-bot@project-id.iam.gserviceaccount.com

If the service account is missing or only has Viewer access, the pipeline may be able to read data but will fail when writing articles, metadata, source validation rows, or trend signals.

Required APIs

In Google Cloud Console, go to:
APIs & Services -> Enabled APIs & services
Confirm these APIs are enabled:

- Google Sheets API

- Google Drive API

Google Sheets API is required for the current database operations. Google Drive API is mostly relevant for Drive upload/file operations and future direct PDF storage, but the codebase already includes Drive helper code, so it should remain enabled.

Where Credentials Are Stored

There are two main places to check credentials:

A) GitHub repository secrets

The collect and trend workflows use:
GOOGLE_CREDENTIALS
GOOGLE_SHEET_ID
GOOGLE_CREDENTIALS should contain the full service account JSON. GOOGLE_SHEET_ID should contain the spreadsheet ID from the Google Sheet URL.

B) Render environment variables

The backend also needs access to:
GOOGLE_CREDENTIALS
GOOGLE_SHEET_ID

These are used when the deployed FastAPI service reads metadata, source lists, trend results, starred summaries, or source validation reports.

Common Failure Symptoms

If the service account or credentials are broken, common errors include:

- pipeline fails during Google Sheets connection
- Failed to open spreadsheet with ID...
- Failed to find worksheets...
- article rows are not saved
- source metadata validation fails immediately
- trend analysis writes no results
- dashboard cannot load source lists or latest results

What to Check First During Google Access Failures

1. Confirm GOOGLE_SHEET_ID points to the correct spreadsheet.
2. Confirm the spreadsheet is shared with the service account email.
3. Confirm the service account has Editor access.
4. Confirm GOOGLE_CREDENTIALS in GitHub Secrets and Render is current.
5. Confirm Google Sheets API is enabled.

6. Confirm Google Drive API is enabled.
7. Check whether the service account key was rotated or deleted.

Credential Rotation Notes

If the service account key is rotated, the new JSON must be updated everywhere the app runs:

- GitHub repository secret: GOOGLE_CREDENTIALS
- Render environment variable: GOOGLE_CREDENTIALS
- local backend/credentials.json, if testing locally

After updating credentials, rerun a small test such as loading source lists, running metadata validation, or triggering a short pipeline test.

8 Maintenance and Operations

Every week, review GitHub Actions logs and Render logs. GitHub Actions logs show pipeline execution details, dependency installation, model loading, extraction errors, PDF generation, and artifact upload. Render logs show backend request handling, auth failures, dispatch failures, status polling, and Google API errors. Watch for repeated HTTP 401/403 responses, empty article counts, unexpected GitHub dispatch errors, and Google Sheets write failures.

Article counts should be checked regularly. A sudden drop can mean source metadata changed, a publication blocked requests, a source list was misconfigured, or article extraction started failing. Summary quality should also be
spot-checked on a small sample because a successful model call can still produce weak output if extraction fed it noisy text.

Monthly, review GitHub Actions runtime and dependency health. The collect workflow has retries and a long timeout because the pipeline can be slow. Run `npm audit`for the frontend and review Python dependency health for the backend. Publications change feeds and HTML structure often, so source validation and pipeline logs should be reviewed together.

Quarterly, rotate long-lived GitHub tokens if used, review the shared app password, audit Google Sheet permissions, and check whether frontend/backend dependencies need major updates. Annually, rotate Google service account credentials, update documentation, and consider archiving historical Google Sheets data if the spreadsheet is becoming too large.

9 Troubleshooting Guide

If the pipeline fails before a GitHub Actions run appears, check Render logs first. This usually means the backend could not dispatch the workflow because of a GitHub token, workflow name, repo, branch/ref, or auth problem. Verify GITHUB_OWNER, GITHUB_REPO, GITHUB_TOKEN, GITHUB_WORKFLOW, and GITHUB_REF, and confirm the workflow exists on that ref.

If the workflow starts but fails inside GitHub Actions, inspect the failing step. Common causes are dependency installation failures, missing Google credentials, invalid sheet ID, import errors, model loading errors, article extraction failures, or PDF generation errors.

If articles do not appear in the dashboard, check the Google Sheet directly. If rows exist in Sheets but not in the UI, inspect frontend API config, auth, CORS, Google Sheets API key, and data-shape assumptions. If rows do not exist, inspect collection, summarization, storage, and source-list selection.

If trend dispatch returns 422 Unexpected inputs provided, the backend is sending inputs that the workflow on the dispatched branch does not define. Confirm the inputs in .github/workflows/trend-analysis.yml, then confirm Render's GITHUB_REF. If GITHUB_REFpoints to an old branch, update it and redeploy Render.

If trend results are empty, check the selected list, active rows, date window, baseline weeks, article count for that period, and host matching between source-list base URLs and article URLs. The trend script intentionally writes a no-trends sentinel when no rows pass filters.

If authentication fails, verify APP_LOGIN_PASSWORD, JWT_SECRET, refresh cookie settings, CORS origins, and browser session state. Clearing browser storage/cookies is a reasonable first step after deployment changes.

10 Known Limitations

A) The current process of adding the new set of publications in the “Add New Category/Topic” page doesn’t take the new set of keywords and the custom prompt, required to run the pipeline.

B) Publications frequently change their website structure, feed paths, and anti-bot behavior without notice. This can cause the scraper to miss articles or fail to extract metadata. The system handles this with RSS and sitemap discovery, browser-like request clients such as curl_cffiand cloudscraper, user-agent rotation, rate limiting, Newspaper3k parsing, BeautifulSoup fallback parsing, and source-level skip behavior. Even with these fallbacks, temporarily failing sources can still cause missed articles.

C) Author extraction remains imperfect. Publications represent bylines in many different ways: JSON-

LD, meta tags, visible text, multiple contributor blocks, dynamically rendered components, or sometimes no clear byline at all. The current system checks structured data, Newspaper3k author fields, and By Name-style regex patterns, but some articles will still return Unknown. Improving this would likely require publication-specific selectors or a journalist-focused named-entity extraction approach.

D) The progress bar is still phase-level rather than truly article-level real time. The backend has websocket support and metadata progress concepts, and the frontend loading screen listens to /pipeline/ws, but the displayed percentage is based on broad phases such as initializing, collecting, and summarizing. For long production runs, users may still feel like the app is stalled. A better version would push granular source/article progress into shared state, potentially Redis, and stream that through websockets.

E) PDF access still depends on GitHub Actions artifacts. The backend can stream the latest artifact ZIP, which is better than sending users manually through GitHub, but the artifact is still a ZIP archive. A cleaner long-term solution would upload PDFs directly to a controlled Google Drive location or have the backend extract/serve the PDF directly.

F) Some dashboard copy is still static or stale. The homepage says it collects from 40+ publications, but the actual number should depend on the selected source list and active row count. Runtime guidance should also reflect the current production behavior, around one and a half hours, rather than older prototype estimates.

G) The frontend .env.examplestill contains legacy VITE_GITHUB_*variables. This is misleading because the current frontend should not receive GitHub tokens. Clean this up so future setup instructions only include current, safe frontend variables.

H) The Drive upload helper in google_storage.pystill contains public sharing behavior if it is ever re-enabled. The current pipeline skips Drive upload and uses GitHub artifact ZIPs, but if the team returns to Drive-based PDF distribution, permissions should be reviewed and restricted according to the organization's access policy.

11 Next Steps

Week 1 (Non-Trivial and urgent):

- Set-up local repo
- Go through the handover documentation and the codebase

(Trivial but urgent):

- Unlink the repo from the forked, TaterTot repo.

- Make sure every pipeline/ process uses the 3.11 version of Python.

- I've used app-revamp branch to push my changes and now, I've merged it with master but it is a few commits behind master. Delete it and use a new branch for your work. Don't directly push new changes to master.

- In the Google sheets, for each article summary, we currently save the date of the pipeline run. Take the published date of the article form the scraper input and save that instead.

(Trivial and not urgent):

- Improve real-time progress. The current progress bar is broad and phase-based. The next version should report source-level or article-level progress. Redis would be useful if backend instances and long-running work do not share memory, but even without Redis, the websocket path should become more granular.

- Make dashboard stats dynamic. Publication count should come from the selected source list's active rows, and runtime guidance should reflect the current production estimate. Static copy makes users misinterpret normal long-running jobs as failures.

- Clean frontend environment documentation. Remove legacy GitHub token variables from frontend `.env.example`and document `VITE_PIPELINE_API_BASE`as the key frontend backend URL.

The coming weeks:

- Add a UI field in the "Add New Category/Topic" page to accept new set of keywords and also a new summarizer prompt, and patch them with the Topic selection dropdown that's on the homepage, so that when users select a topic, it's respective keywords and prompt are used in the pipeline.

- The current "Add New Category/Topic" flow requires users to enter both the publication's base URL and RSS feed URL, while sitemap discovery is already automated. Update this flow so users only provide the base URL, and the system automatically discovers the RSS feed as well.

- Extend the Trend Analysis pipeline's capabilities to come up with pitch-decks based on the identified trends.

12 Future Projects

- Improve the author name extraction by training a custom Named Entity Recognition (NER) model specifically for journalist name extraction could improve accuracy.

- Train/ tune a topic specific model instead of keyword-weight rules to filter the articles.

- Client outreach email drafter.

- There are some more interesting ideas shared by the earlier intern in the Thoughts on Future Projects document. Refer to that doc once you're done with the "The coming weeks" tasks.